IF2211 STRATEGI ALGORITMA
TUGAS BESAR 1

Pemanfaatan Algoritma *Greedy* dalam pembuatan bot permainan Battlecode 2025

BukaPuasaJamSatu
13524121 - Billy Ontoseno Irawan
13524125 - Muhammad Rafi Akbar
13524142 - Rasyad Satyatma

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

Bab 1
Deskripsi Tugas

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi ***greedy*** dalam membuat bot ini.

1.1. Komponen Permainan

Komponen-komponen dari permainan ini antara lain:

1. Objectives

   Objective utama adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai di peta akan memenangkan permainan. Tim juga bisa menang dengan cara menghancurkan semua robot dan menara milik tim lawan.

2. Aturan Tiebreaker

   Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, permainan akan langsung berakhir. Penentuan pemenang dilakukan dengan urutan tiebreaker berikut:

   - Luas area yang sudah diwarnai di peta
   - Jumlah menara sekutu yang masih hidup
   - Total jumlah uang (chips)
   - Jumlah total cat (paint) yang dimiliki semua robot dan menara
   - Jumlah robot sekutu yang masih hidup
   - Jika masih seri, tim dipilih secara acak (uniform random)

3. Peta

   Setiap permainan dimainkan di sebuah peta berbentuk grid persegi panjang 2 dimensi, dengan ukuran antara 20×20 hingga 60×60 petak (inklusif). Titik kiri bawah peta memiliki koordinat (0, 0); koordinat bertambah ke arah Timur (kanan) dan Utara (atas). Lokasi di peta direpresentasikan dengan objek MapLocation yang menyimpan nilai x dan y. Agar tidak ada peta yang menguntungkan salah satu pemain, dijamin bahwa peta bersifat simetris (baik melalui rotasi maupun refleksi).

4. Jarak Pandang dan Pergerakan

   Robot selalu bisa melihat fitur peta dan robot lain dalam jarak hingga sqrt(20), kira kira 4.47 petak. Unit tidak bisa bergerak melewati menara atau robot lain. Peta mungkin memiliki tembok (walls), yang tidak bisa dilewati atau diwarnai. Tembok maksimal hanya 20% dari seluruh peta. Tembok tidak bisa dihapus atau dibangun oleh pemain. Robot juga tidak bisa melewati atau mewarnai reruntuhan (ruins).

5. Tower & Ruins

   Menara juga berfungsi sebagai zona spawn tim dan bisa menyerang unit musuh. Untuk membangun menara, tim harus mewarnai pola khusus di atas reruntuhan (ruins), zona 5×5 yang sudah ditentukan di peta *(lihat section towers untuk selengkapnya)*.

   Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah ada. Reruntuhan tidak bisa dipindah atau dihancurkan. Pusat reruntuhan berjarak minimal 5 petak satu sama lain, dan zona 5×5 di sekitar pusat reruntuhan dijamin bebas tembok. Setiap tim memulai permainan dengan 2 menara: 1 menara cat dan 1 menara uang. Reruntuhan hanya bisa dilihat oleh robot atau menara jika berada dalam radius penglihatan.

   - Money Tower
   - Paint Tower
   - Defense Tower

   Secara default, setiap tim memulai permainan dengan 1 menara penghasil uang dan 1 menara penghasil cat, keduanya sudah di-upgrade ke level 2.

6. Pola Sumber Daya Khusus

   Mewarnai pola khusus tertentu di peta dapat meningkatkan produksi sumber daya tim. Pola ini disebut **Special Resource Pattern (SRP).** Setiap pola aktif yang berhasil diwarnai akan membuat semua menara tambang sekutu mendapatkan +3 sumber daya tambahan per giliran.

Robot bisa menandai pola ini menggunakan fungsi **markResourcePattern(),** lalu menyelesaikannya dengan **completeResourcePattern()** (biaya 200 chips). Setelah pola selesai dibuat, pola tersebut harus tetap utuh (tidak rusak warnanya) selama 50 ronde agar menjadi aktif. Pola ini sama untuk semua peta dan permainan.

7. Cat

Cat adalah sumber daya utama permainan. Digunakan untuk mengklaim wilayah dan menghitung skor. Setiap robot dan menara menyimpan cat secara individu, dengan kapasitas maksimal tergantung tipenya. Robot yang cat-nya sedikit akan mengalami cooldown lebih lama atau bahkan tidak bisa beraksi. Cat bisa diisi ulang dari mopper atau ditarik dari menara penghasil cat.
Setiap tipe robot memiliki serangan berbeda yang bisa:

- Menggunakan cat untuk meletakkan warna di petak, atau
- Menghapus cat milik lawan dari petak.

Robot bisa memilih antara warna utama atau warna sekunder tim untuk mewarnai petak. Kedua warna ini dianggap sama untuk keperluan skor, tapi dibedakan saat mengecek penyelesaian pola cat.

8. Chips

Chips digunakan untuk membangun unit & menara. Chips dihasilkan setiap ronde oleh menara penghasil uang, dan bisa digunakan oleh robot sekutu mana saja setelah dihasilkan. Setiap tim memulai permainan dengan 2500 chips.

9. Robot

Semua robot dapat melakukan aksi seperti bergerak, merasakan (sensing), dan berkomunikasi satu sama lain. Dalam setiap pertempuran, robot milikmu akan berhadapan dengan satu tim musuh. Permainan berjalan berbasis giliran (turn-based) dan dibagi menjadi ronde. Pada setiap ronde, setiap robot mendapat giliran untuk menjalankan kode dan melakukan aksi. Kode yang dijalankan robot menghabiskan bytecode (ukuran sumber daya komputasi, dijelaskan lebih lanjut pada section bytecode limit). Setiap robot hanya memiliki jumlah bytecode tertentu per giliran; jika habis, gilirannya langsung berakhir dan akan dilanjutkan pada giliran berikutnya. Jika robot sudah selesai menjalankan aksinya pada giliran tersebut, ia harus memanggil **Clock.yield()** untuk menunggu giliran berikutnya dimulai.

Semua robot memiliki HP (hitpoints / nyawa). Ketika HP mencapai nol, robot langsung dihapus dari permainan. Setiap robot memiliki ID unik yang diacak dan tidak kurang dari 10.000. Robot hanya dapat berinteraksi dengan lingkungan terdekat melalui sensing, pergerakan, dan kemampuan khusus. Setiap robot menjalankan salinan kode Anda secara independen. Robot tidak bisa berbagi variabel statis (masing-masing punya salinannya sendiri) karena dijalankan di JVM terpisah.

Dua atau lebih robot tidak boleh berada di petak yang sama. Ketika cooldown pergerakan di bawah 10, robot dapat bergerak ke salah satu dari 8 petak tetangga. Semua robot memiliki cadangan cat (paint) dengan kapasitas maksimum tertentu. Semakin rendah cadangan cat, semakin tinggi cooldown pergerakan dan aksi yang dihadapi.

Aturan penalti cat rendah:

- Jika cadangan cat hanya X% penuh (X < 50), maka cooldown meningkat (100 – 2X)%.
- Jika cadangan cat habis sama sekali selama giliran, robot tidak bisa bergerak atau melakukan aksi apa pun (kecuali self-destruct) sampai mendapat cat lagi, dan akan kehilangan 20 HP per ronde.

Robot spawn dengan cadangan cat 100% penuh dan dapat diisi ulang di menara atau oleh mopper. Kehilangan cat saat mengakhiri giliran:

- Di wilayah netral : kehilangan 1 cat
- Di wilayah musuh : kehilangan 2 cat
- Di wilayah sekutu : tidak kehilangan cat

Penalti cat tambahan setiap ronde adalah 1 × jumlah robot sekutu yang bersebelahan (dalam 8 petak sekitar). Pinalti ini berlipat ganda (×2) saat berada di wilayah musuh. Robot dapat di-spawn dalam radius aksi √4 (≈ 2 petak) dari menara sekutu, dengan biaya cat dan chips sesuai jenisnya.

10. Jenis Robot

Soldier

- HP: 250
- Kapasitas cat maks: 200
- Biaya produksi: 200 cat + 250 chips
- Serangan: Menyerang satu petak (radius aksi 3)
    - Jika petak kosong atau sudah ada cat sekutu → mewarnai petak tersebut

    - Menimbulkan 50 damage ke menara musuh (tidak ke robot)
- Biaya serangan: 5 cat
- Cooldown aksi: 10

Splasher

- HP: 150
- Kapasitas cat maks: 300
- Biaya produksi: 300 cat + 400 chips
- Serangan: Menyerang area melingkar (radius 2 dari titik pusat)
    - Titik pusat serangan maksimal 2 petak dari posisi splasher
    - Dalam radius √2 (~1.41) dari pusat → bisa menimpa cat musuh
    - Mewarnai semua petak kosong / sekutu di area
    - Menimbulkan 100 damage ke menara musuh (tidak ke robot)
- Biaya serangan: 50 cat
- Cooldown aksi: 50

Mopper

- HP: 50
- Kapasitas cat maks: 100
- Biaya produksi: 100 cat + 300 chips
- Kemampuan utama:
    1. Mop petak (radius √2)
        - Menghapus cat musuh di petak tersebut
        - Jika ada robot musuh di petak → musuh kehilangan 10 cat, mopper mendapat 5 cat
        - Cooldown: 30
    2. Transfer cat ke robot atau menara sekutu (radius √2)
        - Mentransfer jumlah cat tertentu dari cadangan sendiri
        - Cooldown: 10
    3. Ayunan mop (swing) ke salah satu dari 4 arah utama (atas, bawah, kiri, kanan)
        - Menghapus 5 cat dari masing-masing robot musuh yang berdekatan dalam garis lurus

- Serangan diterapkan dua kali di arah yang sama: → 1 langkah ke arah tersebut → 2 langkah ke arah yang sama
- Maksimal 6 target potensial (3 di langkah 1 + 3 di langkah 2)
- Cooldown: 20

Catatan tambahan: Mopper mendapat penalti cat ganda saat bergerak atau mengakhiri giliran di wilayah musuh.

Ability dan Statistik Robot

Name	Health	Paint Capacity	Paint Cost	Money Cost	Attack Cost	Attack Radius	Attack Damage	Attack Cooldown
Soldier	250	200	200	250	5	3	50 health to towers	10
Splasher	150	300	300	400	50	2	100 health to towers	50
Mopper	50	100	100	300	0	$\sqrt{2}$	-10 paint to robots	30

Untuk informasi lebih lengkap, silahkan buka dokumentasi Battlecode pada link berikut.

11. Towers
    - Prosedur Pembangunan Menara

      Semua menara dibangun dengan prosedur yang sama. Menara hanya bisa dibangun di atas reruntuhan (ruins), setelah pola cat 5×5 tertentu ditempatkan di sekitar reruntuhan tersebut.

      1. Panggil markTowerPattern() untuk secara otomatis menempatkan penanda di area 5×5 sekitar reruntuhan. Penanda ini menunjukkan posisi mana yang harus diwarnai dengan warna utama (primary color).
         - Biaya: 25 cat
      2. Setelah pola sudah diwarnai, robot bisa memanggil completeTowerPattern() untuk memunculkan menara yang ditentukan di lokasi reruntuhan.
         - Menara akan tetap hidup sampai HP-nya mencapai 0, meskipun pola cat di sekitar reruntuhan kemudian dihapus.

- Catatan: Pola menara bisa diselesaikan tanpa memanggil markTowerPattern() terlebih dahulu, asalkan pola cat utama dan sekunder sudah cocok.

- Posisi dan Ukuran Menara

  Menara yang dihasilkan akan menempati blok 1×1 di tengah bentuk spawn. Bentuk berbeda menghasilkan jenis menara berbeda dengan rentang serangan, laju serang, laju penambangan, dan kesehatan yang berbeda (lihat tabel di bawah).

  Level dan Upgrade

  - Menara selalu dibangun di level 1.
  - Bisa di-upgrade ke level lebih tinggi menggunakan upgradeTower() dalam radius sqrt(2) (~1.41 petak).

  Batasan

  - Tidak ada tim yang boleh memiliki lebih dari 25 menara sekaligus.
  - Setiap menara bisa menyimpan 1000 cat.

  Bonus dari Menara Pertahanan

  Setiap menara pertahanan yang hidup meningkatkan kekuatan serangan satu blok (single-block attack) dari semua menara sekutu sebesar:

  - +5 (level 1)
  - +7 (level 2)
  - +9 (level 3)

  Bonus ini tidak memengaruhi serangan area (AoE) menara sekutu.

Name	Build/ Upgrade cost	Attack Range	Single-block Attack Strength	AoE Attack Strength	Base Mining Rate	Health
Money Lv1	1000 chips	3	20	10	20 chips/turn	1000
Lv2	2500 chips	3	20	10	30 chips/turn	1500
Lv3	5000 chips	3	20	10	40 chips/turn	2000
Paint, Lv1	1000 chips	3	20	10	5 paint/turn	1000
Lv2	2500 chips	3	20	10	10 paint/turn	1500
Lv3	5000 chips	3	20	10	15 paint/turn	2000
Defense, Lv1	1000 chips	4	40	20	20 chips/attack*	2000
Lv2	2500 chips	4	50	25	30 chips/attack*	2500
Lv3	5000 chips	4	60	35	40 chips/attack*	3000

12. Serangan

Menara dapat menyerang unit musuh. Menara memiliki dua jenis serangan:

- Serangan satu petak (single-block attack) yang menyerang satu unit musuh di satu petak.
- Serangan area efek (AoE / area of effect) jarak jauh yang memberikan kerusakan kepada semua unit musuh di dalam jangkauannya.

Setiap giliran, satu menara dapat melakukan satu serangan satu petak dan satu serangan area efek.

13. Penambangan

Setiap giliran, menara penghasil uang dan menara penghasil cat akan secara pasif menambang sumber daya.

- Cat disimpan di dalam menara dan dapat diambil oleh robot sekutu.
- Uang (chips) tersedia untuk semua unit sekutu.

Setiap menara memulai dengan 500 cat untuk digunakan spawn berbagai jenis unit. Menara dapat memunculkan robot dalam jarak 2 petak dari posisinya. Jika cat di menara habis, menara tidak akan bisa spawn unit lagi sampai diisi ulang atau menghasilkan cat sendiri. Cat dapat diisi ulang menggunakan cat yang dikumpulkan oleh mopper.

14. Komunikasi

Robot hanya bisa melihat lingkungan di sekitarnya secara langsung dan dikendalikan secara independen oleh salinan kode yang sama, sehingga koordinasi antar-robot sangat sulit. Anda tidak dapat berbagi variabel apa pun di antara mereka; bahkan variabel static pun tidak akan dibagikan karena setiap robot memiliki salinannya sendiri.

Komunikasi dilakukan dengan dua cara:

- Pesan (Messages) Pesan dapat berisi informasi apa saja, tetapi dibatasi hanya 4 byte.
- Penanda (Markers) Penanda menyatakan warna apa yang seharusnya digunakan untuk mewarnai ulang suatu petak, dan penanda ini terlihat oleh semua robot sekutu. Penanda tidak memberikan efek lain apa pun dalam permainan.

Pesan

Robot atau menara yang berada dalam jangkauan satu sama lain dapat saling mengirim pesan. Sebuah robot dianggap "dalam jangkauan" (in range) sebuah menara jika memenuhi dua syarat sekaligus:

- Jaraknya ke menara tersebut ≤ sqrt(20) (~4.47 petak), dan
- Robot tersebut terhubung dengan menara melalui jalur cat sekutu (artinya robot bisa bergerak sampai ke menara hanya dengan berjalan di petak-petak yang sudah memiliki cat milik timnya sendiri).

The robots and towers who are connected by paint can talk, but the robot who is not on paint cannot talk to the tower.

Setiap robot dan menara menyimpan semua pesan yang diterima dalam 5 ronde terakhir di dalam buffer-nya. Setelah 5 ronde berlalu, pesan tersebut akan dihapus secara otomatis.

Batas Pengiriman Pesan per Ronde adalah sebagai berikut

- Robot: maksimal 1 pesan per ronde
- Menara: maksimal 20 pesan per ronde

Setiap pesan terdiri dari:

- Satu bilangan bulat 32-bit (integer)
- Nomor ronde saat pesan dikirim
- ID robot yang mengirim pesan

Menara memiliki kemampuan tambahan untuk melakukan broadcast (siaran) pesan ke menara sekutu lain yang berada dalam jarak sqrt(80) (~8.94 petak).

- Menara tidak perlu terhubung melalui jalur cat untuk melakukan broadcast ini.
- Setiap kali broadcast dilakukan, itu dihitung sebagai satu pesan terhadap batas 20 pesan per ronde menara.
- Kemampuan ini memungkinkan menara memperluas jangkauan komunikasi jarak jauh antar-menara.

Penanda

Penanda adalah informasi yang bisa ditempatkan atau dihapus di peta oleh robot. Penanda hanya memiliki dua jenis:

- Menandai dengan warna sekunder
- Menandai dengan warna utama

Penanda hanya terlihat oleh robot sekutu dan akan tetap ada di petak tersebut sampai dihapus atau diganti dengan penanda lain oleh sekutu. Selain itu, penanda hanya dapat ditempatkan pada ubin yang dapat dicat

Penanda dapat diletakkan dengan cara

- Secara manual: robot dapat menempatkan penanda di petak yang berjarak ≤ sqrt(2) (~1.41 petak) dari posisi robot saat ini. Akan dijelaskan lebih lanjut pada section Action
- Secara otomatis melalui fungsi pola:
    - markTowerPattern() → menandai pola 5×5 di sekitar reruntuhan dengan penanda yang sesuai untuk membangun menara
    - markResourcePattern() → menandai pola 5×5 untuk Special Resource Pattern (SRP)

15. Action and Cooldown

Robot melakukan berbagai aksi untuk berinteraksi dengan dunia permainan. Beberapa aksi tidak bisa dilakukan berkali-kali dalam satu giliran atau dalam waktu singkat karena terikat cooldown. Berikut daftar aksi utama:

- Menyerang/mewarnai

  Menyerang dan mewarnai dilakukan dengan fungsi yang sama. Aksi ini hanya bisa dilakukan jika action cooldown robot < 10. Action cooldown robot dapat dicek dengan fungsi isActionReady(). Dilakukan dengan memanggil metode attack() yang akan menaargetkan satu petak yang dapat dilihat oleh robot tersebut. Efeknya tergantung tipe robot (seperti yang dijelaskan di bagian Robot): memberikan damage atau mewarnai petak. Setelah dilakukan, action cooldown robot bertambah sesuai nilai yang ditentukan oleh tipe robot.
- Bergerak (moving)

  Hanya bisa dilakukan jika movement cooldown < 10. Movement cooldown dapat dicek dengan isMovementReady(). Petak tujuan harus kosong (tidak ditempati unit lain) dan bisa dilewati (passable). Dilakukan dengan memanggil move() dengan arah yang

diinginkan. Setelah bergerak, movement cooldown bertambah 10. Untuk memeriksa apakah gerakan valid, gunakan canMove().

- Menandai (marking)

  Semua robot bisa menandai petak tempat robot sedang berdiri, atau Menghapus penanda sekutu dari petak tempat robot berdiri. Biaya dari aksi ini adalah 1 cat. Aksi ini tidak menambah action cooldown sama sekali. Aksi dapat dilakukan dengan mark() atau removeMark(). Penanda tidak memengaruhi kepemilikan wilayah (tidak membuat petak menjadi wilayah sekutu). Penanda hanya bisa dirasakan oleh robot sekutu melalui senseNearbyMapInfos(). Penanda secara umum tersebut hanya boleh ditempatkan pada petak yang dapat diwarnai (tidak ada tembok atau reruntuhan). Selain itu, terdapat Fungsi khusus penanda, yakni markTowerPattern() dan markResourcePattern()menggunakan 25 cat dan menandai area 5×5 dengan pola yang sesuai.

- Mentransfer Cat (Transferring)

  Hanya dapat dilakukan oleh robot mopper, dan hanya dapat dilakukan jika action cooldown < 10 (dapat dicek dengan isActionReady()). Aksi ini dapat dilakukan dengan memanggil prosedur transferPaint(). Jarak maksimal: √2 (~1.41 petak). Mentransfer jumlah cat tertentu ke target (robot atau menara sekutu). Jika jumlah yang diberikan negatif, maka mopper justru mengambil cat dari target. Setelah dilakukan, action cooldown bertambah sesuai nilai yang disebutkan di deskripsi mopper.

- Menarik cat dari menara (withdrawing)

  Semua robot bisa menarik cat dari menara sekutu untuk mengisi ulang cadangan cat mereka. Memerlukan action cooldown < 10. Setelah dilakukan, action cooldown bertambah 10. Jarak maksimal: √2 (~1.41 petak). Dilakukan dengan transferPaint() menggunakan nilai negatif untuk menunjukkan jumlah yang ingin ditarik.

- Spawn Robot (spawning)

  Hanya bisa dilakukan jika action cooldown menara < 10 (dicek dengan isActionReady()). Dilakukan dengan buildRobot(). Mengurangi biaya robot (cat + chips) dari stok menara. Memunculkan satu robot tipe tertentu di petak yang ditentukan. Setelah dilakukan, action cooldown menara bertambah 10.

- Berkomunikasi (communitating)

  Robot menggunakan sendMessage() untuk mengirim pesan ke menara sekutu dalam jangkauan. Menara menggunakan sendMessage() untuk mengirim pesan ke robot sekutu dalam jangkauan. Robot menggunakan readMessages() untuk membaca pesan baru yang diterima dalam 5 ronde terakhir dari buffer. Kedua metode ini tidak menambah action

cooldown. Syarat komunikasi: kedua unit harus berada di petak yang terhubung melalui jalur cat sekutu.

- Menghancurkan Diri Sendiri (Disintegrating)

  Robot apa pun bisa memanggil disintegrate(). Menghancurkan robot tersebut secara instan. Tidak ada cooldown khusus untuk aksi ini.

1.2 Batasan Program

- Bytecode

Jumlah komputasi yang boleh dilakukan oleh robot pada setiap giliran sangat terbatas. Bytecode menjadi ukuran yang praktis untuk menghitung komputasi di bahasa pemrograman seperti Java, di mana satu bytecode Java kira-kira setara dengan satu operasi dasar seperti pengurangan atau pengambilan field, dan satu baris kode biasanya mengandung beberapa bytecode (untuk detail lebih lanjut bisa dilihat di tautan tersebut). Karena bytecode merupakan bagian dari kode yang sudah dikompilasi, program yang sama akan selalu menghasilkan bytecode yang identik dan karena itu membutuhkan jumlah komputasi yang persis sama pada input yang sama. Keunggulan ini sangat membantu karena kita bisa menghindari penggunaan waktu sebagai ukuran komputasi, yang sering menimbulkan masalah seperti perilaku tidak deterministik. Dengan adanya batas bytecode, menjalankan ulang pertandingan yang sama antara bot yang sama akan selalu menghasilkan hasil yang persis identik, fitur yang sangat berguna untuk proses debugging.

Setiap ronde, setiap robot mengambil gilirannya secara berurutan. Jika sebuah robot mencoba melebihi batas bytecode-nya (biasanya terjadi tanpa disengaja, misalnya karena ada loop yang terlalu besar atau hal serupa), komputasinya akan langsung dihentikan sementara dan kemudian dilanjutkan kembali tepat dari titik itu pada giliran ronde berikutnya. Kode akan berjalan lagi dengan normal, tetapi hal ini bisa menimbulkan masalah. Misalnya, robot sempat memeriksa apakah sebuah petak masih kosong, lalu komputasinya terpotong, robot lain bergerak dulu, dan ketika giliran robot itu dilanjutkan ia mencoba bergerak ke petak yang sekarang sudah ditempati orang lain. Untuk menghindari situasi seperti ini, gunakan fungsi Clock.yield() untuk mengakhiri giliran robot secara sengaja. Fungsi ini akan menghentikan komputasi tepat di tempat yang kamu inginkan, lalu melanjutkannya dari baris kode berikutnya pada giliran ronde selanjutnya.

Batas bytecode untuk semua robot adalah 17500, sedangkan batas bytecode untuk semua menara adalah 20000.

- Package

  Pemain dapat menggunakan kelas dari paket mana pun yang tercantum dalam AllowedPackages.txt, kecuali kelas yang tercantum dalam DisallowedPackages.txt. File-file ini dapat ditemukan di sini.

  Selain itu, pembatasan berikut juga berlaku:

  - `Object.wait`, `Object.notify`, `Object.notifyAll`, `Class.forName`, dan `String.intern` tidak diperbolehkan.
  - `java.lang.System` hanya mendukung `out`, `arraycopy`, dan `getProperty`.
  - Selain itu, `getProperty` hanya boleh digunakan untuk mengambil properti dengan nama yang diawali dengan "`bc.testing.`".
  - `java.io.PrintStream` tidak boleh digunakan untuk membuka file.

  Perlu diperhatikan bahwa pelanggaran terhadap salah satu pembatasan di atas akan menyebabkan robot meledak saat dijalankan, meskipun file sumber berhasil dikompilasi tanpa masalah.

- Memory

  Robot harus menjaga penggunaan memori tetap wajar. Jika sebuah robot menggunakan lebih dari 8 MB heap space selama turnamen atau pertandingan scrimmage, robot tersebut dapat meledak.

- Execution Time

  Robot harus menjaga waktu eksekusinya tetap wajar. Walaupun penghitungan bytecode memang dimaksudkan untuk tujuan ini, beberapa penggunaan Java yang sangat canggih dapat menyebabkan robot berjalan sangat lambat meskipun masih berada dalam batas bytecode.

  Untuk mengurangi beban pada server, diberlakukan batas total waktu eksekusi per tim. Waktu ini dihitung secara kumulatif untuk semua robot dalam satu pertandingan, dan akan di-reset antar pertandingan dalam satu game.

  Jika sebuah tim melebihi batas ini, tim tersebut akan otomatis menyerah dan kalah dalam game. Nilai batas ini dapat diakses melalui konstanta game, dan Anda dapat mengecek berapa banyak waktu yang telah berlalu melalui kelas Clock.

  Hal ini seharusnya tidak mempengaruhi sebagian besar pemain. Jika Anda mendapati bahwa Anda mencapai batas tersebut, kemungkinan ada optimasi yang perlu dilakukan pada kode Anda.

Bab 2

Landasan Teori

2.1. Dasar Teori

Algoritma greedy adalah algoritma yang mengikuti heuristik pemecahan masalah dengan membuat pilihan optimal lokal pada setiap tahap dan berharap akan berakhir dengan optimum global. Kegunaan utama dari algoritma greedy adalah untuk menemukan solusi optimal dalam persoalan optimasi dengan cepat. Pendekatan ini sangat berguna dalam banyak kasus di mana kita perlu memaksimalkan atau meminimalkan sesuatu dengan cara yang efisien.

Elemen-elemen algoritma greedy:

1. Himpunan kandidat, C : berisi kandidat yang akan dipilih pada setiap langkah
2. Himpunan solusi, S : berisi kandidat yang sudah dipilih
3. Fungsi solusi: menentukan apakah himpunan kandidat yang dipilih sudah memberikan solusi
4. Fungsi seleksi (selection function): memilih kandidat berdasarkan strategi greedy tertentu. Strategi greedy ini bersifat heuristik.
5. Fungsi kelayakan (feasible): memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam himpunan solusi (layak atau tidak)
6. Fungsi obyektif : memaksimumkan atau meminimumkan

Dengan menggunakan elemen-elemen di atas, maka dapat dikatakan bahwa algoritma greedy melibatkan pencarian sebuah himpunan bagian, S, dari himpunan kandidat, C; yang dalam hal ini, S harus memenuhi beberapa kriteria yang ditentukan, yaitu S menyatakan suatu solusi dan S dioptimisasi oleh fungsi obyektif.

2.2. Cara Kerja Program

Bot dalam Battlecode 2025 bekerja dengan menjalankan sebuah loop setiap giliran permainan (turn). Pada setiap turn, bot terlebih dahulu membaca kondisi lingkungan di sekitarnya menggunakan fungsi sensing, seperti mendeteksi posisi musuh, teman, atau sumber daya di peta. Setelah itu, bot memproses informasi tersebut untuk menentukan keputusan atau aksi yang paling sesuai, misalnya menyerang musuh, bergerak menuju target, mengumpulkan sumber daya, atau menjelajah peta. Keputusan ini biasanya diatur menggunakan logika seperti state machine atau aturan kondisi tertentu. Salah satu metode sederhana yang sering digunakan untuk pergerakan adalah algoritma greedy, yaitu dengan memilih langkah yang paling mendekati target. Setelah keputusan diambil, bot menjalankan aksi yang dipilih dan mengakhiri turn sebelum loop dimulai kembali pada turn berikutnya. Untuk menjalankan bot, developer biasanya menggunakan scaffold resmi Battlecode, kemudian melakukan kompilasi program (misalnya dengan Gradle) dan menjalankan simulasi pertandingan secara lokal untuk menguji strategi serta melihat hasilnya melalui visualizer pertandingan dalam client.

Bab 3

Aplikasi Strategi Greedy

3.1. Proses Mapping Persoalan

Battlecode 2025 merupakan suatu permainan strategi *real-time* yang dimainkan oleh program yang dibuat seseorang, dan tidak "dimainkan" dalam definisi tradisional. Program yang dibuat oleh pemain dijalankan untuk tiap robot yang ada di *battlefield*, batasan-batasan yang muncul sebab tiap robot menjalankan kode yang sama membutuhkan penyelesaian yang kreatif, untuk mengakomodasikan situasi *battlefield* yang sangat beragam. Penyelesaian inilah yang dijadikan sebagai kompetisi untuk mencari bentuk solusi seperti apa yang paling efektif dalam berstrategi dalam manajemen sumber daya, beradaptasi melawan berbagai jenis lawan yang bervariasi, sembari tetap fokus untuk memenangkan pertandingan.

Himpunan kandidat aksi untuk robot-robot Battlecode 2025 dapat dibedakan berdasarkan jenis robot yang sedang digunakan. Robot Soldier memiliki aksi untuk berpindah petak, menyerang suatu petak, membangun Tower di suatu Ruin, dan menyelesaikan Special Resource Pattern (SRP). Splasher memiliki aksi-aksi seperti berpindah petak, menyerang suatu area dengan Splash, dan menyelesaikan SRP. Mopper memiliki aksi untuk berpindah petak, melakukan Mop Swing, menghapus cat musuh di suatu petak, dan memberikan cat kepada robot lain.

Himpunan solusi dalam konteks Battlecode 2025 dapat dibilang sebagai urutan aksi yang diambil semua robot selama berjalannya permainan. Aksi-aksi yang diambil tiap robot secara ideal adalah aksi-aksi yang selalu berkontribusi ke kemenangan program. Fungsi kelayakan yang menentukan apakah suatu aksi dapat dilakukan berdasarkan kondisi-kondisi seperti *cooldown,* jumlah cat, dan faktor-faktor lainnya dibuat untuk menentukan apakah suatu aksi dapat dilakukan pada saat itu. Fungsi seleksi menentukan kandidat mana saja yang layak untuk diambil. Serta fungsi objektif yang memaksimalkan kondisi kemenangan, dalam hal ini mengecat 70% dari semua petak dalam peta atau sekadar menguasai lebih banyak petak dibanding lawan pada *tiebreaker*.

3.2. Eksplorasi Alternatif Solusi

Pendekatan yang beragam dapat dipertimbangkan, berdasarkan perspektif pembuat program. Beberapa solusi dengan pendekatan *greedy* yang dapat dipilih, masing-masing dengan *heuristic* yang berbeda-beda sebagai berikut.

a. Pendekatan 1 "billbot"

   Setiap tipe unit dalam bot menggunakan pendekatan *greedy* yang sama. Meskipun strateginya konsisten di seluruh program, heuristik spesifiknya berbeda untuk Soldier, Mopper, Splasher, dan Tower.

   Soldier berfokus pada ekspansi dan pembangunan Tower. Heuristik utama mereka adalah mencari Ruin terdekat yang dapat digunakan untuk membangun Tower. Soldier memindai ubin di dekatnya, memeriksa mana yang berisi ruin, dan memilih ruin dengan jarak

terkecil dari lokasi saat ini. Jika Ruin ditemukan, Soldier bergerak ke arahnya dan mulai mengecat pola yang diperlukan untuk membangun Tower. Jika tidak ada Ruin yang tersedia, Soldier beralih ke perilaku tempur dengan menyerang Tower musuh jika berada dalam jangkauan. Ketika tidak ada Ruin maupun musuh di dekatnya, Soldier berfokus pada penyebaran cat pada ubin kosong, terutama ketika mereka masih berada dalam radius tertentu dari *spawn location*

Splasher menyerang area dengan konsentrasi cat musuh yang tinggi. Heuristik greedy mereka mengevaluasi lokasi serangan yang mungkin dalam area tetap di sekitar unit. Untuk setiap ubin target potensial, Splasher menghitung skor berdasarkan seberapa banyak cat musuh yang ada dalam radius *splash* di sekitarnya. Ubin yang dicat musuh akan mendapatkan skor tinggi, sedangkan ubin kosong akan mendapatkan skor kecil. Splasher kemudian memilih lokasi dengan skor tertinggi dan menyerang jika skor tersebut melewati ambang batas tertentu. Jika tidak ada target yang bagus, Splasher akan mencari ubin terdekat yang berisi cat musuh dan bergerak ke arahnya. Jika tidak ditemukan cat musuh sama sekali, robot Splasher beralih ke perilaku eksplorasi dan bergerak menuju target eksplorasi yang dihasilkan. Pada intinya, heuristik robot Splasher adalah memaksimalkan *splash* dengan menargetkan area dengan cat musuh terbanyak di dekatnya.

Mopper berfokus pada penghapusan cat musuh dari peta. Heuristik mereka lebih sederhana daripada Splasher karena mereka hanya perlu menemukan dan membersihkan ubin yang dicat musuh. Mopper memindai ubin peta di dekatnya dan mengidentifikasi ubin yang berisi cat musuh. Kemudian, Mopper memilih ubin terdekat yang dicat musuh dan bergerak ke arahnya. Jika ubin berada dalam jangkauan serangan, Mopper segera menghilangkan cat. Selain itu, Mopper dapat menggunakan serangan ayunan pel ke arah target. Jika tidak ada cat musuh yang terdeteksi di dekatnya, Mopper mulai menjelajahi peta.

Tower bertindak sebagai pusat produksi dan peningkatan. Heuristik mereka berfokus pada manajemen sumber daya dan pembuatan unit. Tower pertama-tama memeriksa apakah sumber daya (chip) cukup untuk meningkatkan dirinya sendiri. Jika ya, dan jika kondisi tertentu terpenuhi (seperti memiliki cukup unit sekutu di dekatnya atau memiliki kelebihan sumber daya) ia segera melakukan peningkatan. Jika tidak, Tower memutuskan unit mana yang akan dibuat secara acak. Di awal permainan, Tower memiliki kemungkinan yang lebih tinggi untuk memproduksi Soldier, sementara di akhir permainan kemungkinan untuk memproduksi Splasher dan Mopper lebih tinggi. Tower kemudian mencoba untuk membuat unit yang dipilih di arah mana pun yang berdekatan jika memungkinkan.

b. Pendekatan 2 “laliro”
Pendekatan dengan menggunakan algoritma *greedy* berbasis prioritas berdasarkan sumber daya dengan pemindaian lokal. Aksi tiap robot sudah memiliki prioritas yang terstruktur, tetapi dapat berubah berdasarkan hal-hal di sekitar robot.

Soldier memprioritaskan pembangunan Tower dan klaim SRP terdekat, diikuti dengan eksplorasi peta berdasarkan petak-petak di sekitarnya, tidak hanya dengan navigasi secara acak. Prioritas petak yang dipilih didasarkan kondisi petak saat itu juga, dengan skor 15 jika petak dicat musuh, 10 jika belum dicat, dan 0 jika memiliki cat sendiri, dikurangi jarak dari robot.

Splasher menentukan target untuk di-*splash* berdasarkan jenis petak dan jarak dari robot. Target yang berjarak lebih dekat diprioritaskan dengan menghitung jumlah petak yang kosong dan memiliki cat musuh, sedangkan petak yang lebih jauh hanya menghitung petak kosong. Perilaku ini memuat kondisi sekitar robot untuk memastikan bahwa *ability splash* digunakan secara optimal. Selain itu, pergerakan Splasher cenderung ke arah dengan petak kosong paling banyak.

Logika robot Mopper cenderung untuk memberikan cat milik sendiri ke robot lainnya. Mopper melakukan *mop* ke petak yang memiliki cat musuh yang dalam jangkauan diri sendiri. Pergerakan Mopper menuju petak-petak dengan cat musuh, dan mengikuti robot lainnya.

Terakhir, Tower melakukan *spawn* robot yang mana jenis robot yang di-*spawn* bergantung pada *round*. Rasio jenis robot yang bisa di-*spawn* cenderung melakukan *spawn* Soldier di ronde-ronde awal, dan *spawn* Splasher dan Mopper pada ronde-ronde akhir. Lokasi *spawn* robot juga cenderung mendekati tengah peta, dan tidak dipilih secara acak. Tower juga melakukan *targeting* ke robot lawan dengan *HP* terendah dalam jangkauannya.

c. Pendekatan 3 “MyBotGweh”
Heurestik yang dipakai pada bot ini berfokus pada Ekspansi Wilayah dan juga Attack musuh dengan lowest HP. Pada pendekatan ini, Soldier fokus pada ekspansi wilayah dan juga combat, dengan heurestik utama yang mencari area kosong atau ada paint musuh terdekat. Soldier memeriksa area sekitarnya, dan memberikan prioritas area cat lawan dibanding area kosong, agar dapat mengubah nya menjadi area cat sekutu. Jika ada tower musuh dalam range, Soldier akan langsung menyerang sebelum melanjutkan ekspansi. Ketika semua daearah sekitar Soldier sudah diwarnai, Soldier akan melakukan eksplorasi dengan memprioritaskan area yang jaraknya paling jauh dari posisi spawn Soldier..

Splasher berfokus pada coverage area luas, dengan Heurestik greedy yang digunakan yaitu mengevaluasi setiap area dalam radius 2 sebagai daerah dengan potensi sebagai titik pusat melakukan splash. Dengan membuat perhitungan score yang memprioritaskan area cat musuh dibanding area kosong, splasher memilih titik pusat splash yang memiliki “worth” paling banyak sehingga mencegah adanya pemborosan paint (cat musuh > area kosong). Jika tidak ada target splash yang bagus, maka Splasher akan mencoba untuk bergerak ke area kosong atau area cat musuh terdekat.

Mopper berfokus pada memberi support cat ke unit team dan juga membersihkan cat musuh, dimana Mopper cenderung mengikuti Soldier. Apabila tidak ada Soldier, Mopper memprioritaskan untuk bergerak ke area cat musuh terdekat apabila ada. Heurestik Mopper menghitung jumlah musuh di setiap arah dan memilih arah dengan musuh paling banyak.

Tower bekerja sebagai pusat produksi unit, pada bot ini, PaintTower memiliki rasio spawn 3-1-1 untuk Soldier,Mopper, dan Splasher per 5 spawn, dan hanya spawn kalo minimal memiliki 150 paint. MoneyTower beradaptasi sesuai ukuran map dan melakukan spawn. Kedua tower aktif menyerang musuh yang ada dalam rang dan HP terendah.

3.3. Analisis Efisiensi dan Efektivitas

A. Bot "billbot"

a. Eksplorasi

Dalam hal kompleksitas waktu, algoritma ini berulang melalui serangkaian delapan arah tetap yang konstan. Namun, dalam setiap iterasi, algoritma memanggil fungsi untuk mendeteksi robot terdekat. Jika ada n robot dalam radius penginderaan, menghitungnya membutuhkan pemindaian daftar tersebut. Oleh karena itu, biaya utama berasal dari operasi penginderaan ini. Karena loop luar berjalan sejumlah konstan kali (8), kompleksitas total tumbuh secara proporsional dengan jumlah robot terdekat yang terdeteksi. Akibatnya, kompleksitas waktu algoritma ini adalah O(n), di mana n mewakili jumlah robot terdekat yang terdeteksi selama evaluasi.

b. Mencari Ruin terdekat (Soldier)

Algoritma pemilihan Ruin terdekat digunakan oleh Soldier untuk menemukan Ruin di peta yang berpotensi digunakan untuk membangun Tower. Soldier mendeteksi ubin peta terdekat dan menyimpannya dalam sebuah daftar. Kemudian, unit Soldier mengulangi setiap ubin untuk memeriksa apakah ubin tersebut berisi Ruin. Jika ya, algoritma memeriksa apakah sudah ada Tower di lokasi tersebut. Jika tidak ada Tower yang menempati Ruin, algoritma menghitung jarak kuadrat antara lokasi unit saat ini dan lokasi Ruin. Ruin dengan jarak terkecil dipilih sebagai kandidat terbaik untuk membangun Tower.

Kompleksitas waktu bergantung pada jumlah ubin terdekat yang dideteksi. Misalkan operasi pendeteksian mengembalikan n ubin. Algoritma melakukan satu perulangan yang memeriksa setiap ubin tepat sekali dan melakukan operasi waktu konstan di dalam perulangan, seperti memeriksa properti dan menghitung jarak. Karena tidak ada perulangan bersarang yang terlibat, waktu eksekusi tumbuh secara linier dengan jumlah ubin yang dideteksi. Oleh karena itu, kompleksitas waktu algoritma ini adalah O(n), di mana n adalah jumlah ubin yang dikembalikan oleh fungsi pendeteksian.

c. Mencari target splash (Splasher)

Meskipun terdapat perulangan bersarang, ukuran perulangan ini tetap. Area pencarian luar selalu berisi 49 ubin (7 x 7), dan evaluateSplash selalu memeriksa tepat 9 ubin (3 x 3). Karena nilai-nilai ini tidak berubah seiring bertambahnya ukuran peta atau seiring berjalannya permainan, jumlah operasi yang dilakukan oleh algoritma tetap konstan. Dalam notasi Big-O, pekerjaan konstan direpresentasikan sebagai O(1). Oleh karena itu, algoritma penargetan percikan berjalan dalam waktu konstan terlepas dari ukuran peta atau jumlah unit.

d. Mencari cat musuh terdekat (Mopper)

Algoritma deteksi cat musuh terdekat digunakan oleh unit Mopper untuk menemukan ubin yang dicat musuh yang harus dibersihkan. Mopper memulai dengan mendeteksi ubin peta terdekat dan menyimpannya dalam sebuah daftar. Kemudian, Mopper memindai setiap ubin dalam daftar ini untuk menentukan apakah ubin tersebut mengandung cat musuh. Jika ya, algoritma menghitung jarak kuadrat dari posisi Mopper saat ini ke ubin tersebut. Ubin dengan jarak minimum dipilih sebagai target. Setelah mengidentifikasi lokasi cat musuh terdekat, Mopper akan menyerangnya jika memungkinkan atau bergerak ke arahnya

Kompleksitas waktu bergantung pada berapa banyak ubin yang dikembalikan oleh operasi pendeteksian. Jika n ubin dideteksi, algoritma harus memeriksa setiap ubin tepat sekali. Setiap iterasi melakukan operasi waktu konstan seperti memeriksa jenis cat dan menghitung jarak. Karena hanya ada satu loop dan tidak ada loop bersarang, waktu eksekusi meningkat secara linier dengan jumlah ubin yang diperiksa. Akibatnya, kompleksitas waktu algoritma ini adalah O(n), di mana n adalah jumlah ubin terdekat yang dideteksi oleh Mopper.

B. Bot “laliro”

a. Eksplorasi

Eksplorasi diimplementasikan dengan mengambil salah satu petak dalam peta secara acak, dimana semua bot mendatangi petak itu bersama-sama. Tidak adanya rekurensi maupun iterasi membuat algoritma eksplorasi memiliki kompleksitas waktu O(1).

b. Pencarian Struktur (Ruin & Tower)

Pencarian Tower dan Ruin sama-sama memiliki kompleksitas waktu O(n), dimana N adalah jumlah Ruin untuk pencarian Ruin, dan jumlah robot untuk pencarian Tower. Kedua algoritma pencarian bekerja dengan pertama mencari semua struktur di dekat robot, lalu melakukan iterasi untuk tiap struktur untuk mencari mana yang paling dekat.

c. Pencarian Target (Splasher)

Target terbaik didapatkan oleh Splasher dengan cara mengambil semua petak dalam jangkauan Splasher, lalu mencari petak mana yang menghasilkan nilai maksimal untuk dijadikan target. Perulangan yang *nested* menjadikan algoritma pencarian target Splasher memiliki kompleksitas waktu $O(N^2)$, karena jumlah petak yang diambil tidak tetap.

d. Pencarian Target (Mopper)

Mirip dengan pencarian target Splasher, Mopper juga melakukan iterasi petak dalam jangkauan Mopper untuk dijadikan kandidat target. Semua petak dalam jangkauan lalu diiterasi dan petak pertama yang bisa diserang dan dimiliki musuh akan diserang. Iterasi sekali membuat kompleksitas waktu algoritma pencarian target Mopper O(N).

C. Bot "MyBotGweh"

a. Eksplorasi

Algoritma eksplorasi digunakan ketika tidak ada target paint dalam vision radius. Algoritma ini bekerja dengan memilih satu dari 9 titik target yang sudah ditentukan berdasarkan ukuran map. Titik-titik ini diurutkan secara descending berdasarkan jarak. Setelah urutan ditentukan, robot bergerak menuju titik target berikutnya yang cukup jauh dari posisi saat ini.Untuk proses pengurutan awal, insertion sort pada 9 elemen berjalan dalam $O(9^2) = O(1)$ karena ukurannya konstan. Untuk pemilihan target berikutnya, algoritma melakukan iterasi maksimal 9 elemen untuk menemukan titik yang valid, sehingga kompleksitasnya O(1). Karena semua operasi bekerja pada array berukuran tetap (9 elemen), kompleksitas total algoritma eksplorasi adalah O(1).

b. Mencari Target terdekat (Soldier)

Algoritma pemilihan target paint digunakan oleh Soldier untuk menemukan petak kosong atau bercat musuh yang paling optimal untuk dicat. Soldier mendeteksi semua petak di sekitarnya menggunakan senseNearbyMapInfos() dan menyimpannya dalam sebuah daftar. Kemudian Soldier melakukan iterasi pada setiap petak untuk memeriksa apakah petak tersebut passable dan bercat musuh atau kosong. Untuk setiap kandidat, algoritma menghitung skor berdasarkan prioritas (cat musuh = 2, kosong = 1) dan jarak. Pada large map, algoritma juga memperhitungkan jumlah teman di sekitar petak kandidat untuk menghindari crowding.

Kompleksitas waktu bergantung pada jumlah petak yang dikembalikan oleh operasi pendeteksian. Misalkan operasi pendeteksian mengembalikan n petak. Algoritma melakukan satu perulangan yang memeriksa setiap petak tepat

sekali dan melakukan operasi waktu konstan di dalam perulangan seperti memeriksa tipe cat dan menghitung jarak. Karena tidak ada perulangan bersarang yang terlibat, waktu eksekusi tumbuh secara linier dengan jumlah petak yang dideteksi. Oleh karena itu, kompleksitas waktu algoritma ini adalah O(n), di mana n adalah jumlah petak yang dikembalikan oleh fungsi pendeteksian.

Masih terdeteksi masalah dimana terkadang bot menerapkan unusual behaviour seperti diam di tempat dan tidak bergerak dan muter-muter di tempat yang sama.

c. Mencari target splash (Splasher)

Algoritma pemilihan target mengevaluasi setiap area dalam radius 2 dari posisi Splasher sebagai kandidat titik pusat splash area, dimana untuk setiap kandidat algoritma menghitung score dengan mengecek area dalam radius $\sqrt{2}$ di sekitarnya, dengan area kosong memiliki score 2, dan area cat musuh score 3, karena kita ingin memprioritaskan area musuh. Dan Splasher akan memilih titik pusat tertinggi.

Terdapat perulangan untuk mengevaluasi kandidat titik pusat dan juga menghitung score, karena kedua perulangan ini bersifat tetap, dan tidak memerlukan informasi seperti ukuran map dll, jumlah operasi nya konstan, yang menjadikan kompleksitas nya O(1)

d. Transfer & Swing (Mopper)

Algoritma transfer paint ke team bekerja dengan mendeteksi semua robot teman di sekitarnya, lalu filter yang ada dalam radius $\sqrt{2}$ dan memiliki total paint rendah. Mopper akan memilih unit dengan paint paling rendah untuk mentranfer. Algoritma melakukan perulangan yang memeriksa tiap robot sekali dengan operasi waktu konstan. Kompleksitas Algoritma = O(n)

Algoritma untuk swing menentukan arah terbaik untuk menyerang area, dimana Mopper mengecek arah timur, barat, selatan, utara, dan memilih arah dengan jumlah musuh paling banyak. Terdapat nested loop yang berjalan 4 kali, dan juga perulangan untuk memeriksa musuh yang terdeteksi. Maka Kompleksitas total adalah O(4 x m) dimana m adalah jumlah robot musuh yang dapat dideteksi mopper)

3.4. Pemilihan Strategi Greedy "MyBotGweh"

Bot MyBotGweh terpilih sebagai karena memiliki pendekatan yang paling adaptif dari ketiga pendekatan yang disebutkan.

a. Adaptasi Ukuran Peta

Battlecode 2025 memiliki ukuran peta yang beragam, mulai dari 20x20 petak hingga 60x60 petak. Pendekatan MyBotGweh secara cerdas mengubah rasio *spawn* unit berdasarkan ukuran peta sekarang, fokus Splasher untuk peta yang lebih besar, dan Soldier dan Mopper untuk peta yang lebih kecil

b. Pengangan Kondisi Terjebak

Algoritma pencarian jalur MyBotGweh memiliki deteksi ketika robot tidak berpindah setelah beberapa ronde, dan memaksakan pergerakan acak untuk robot yang dideteksi sebagai terjebak agar bisa mencari jalur yang baru.

c. Eksplorasi Berbasis Zona

Peta yang dibagi menjadi zona 3x3 oleh program MyBotGweh, memaksimalkan penguasaan peta dengan mendistribusikan robot-robot ke semua zona. Pendekatan ini memaksimalkan hasil dengan mengimplementasikan algoritma sederhana.

Bab 4

Implementasi dan Pengujian

4.1. Implementasi

a. Bot “billbot”

```
GREEDY_MOVE_TOWARD(target)

    currentLocation ← robot location
    bestDirection ← null
    bestScore ← infinity

    FOR each direction in 8 possible directions DO

        IF robot cannot move in direction
            CONTINUE

        newLocation ← currentLocation + direction

        score ← distanceSquared(newLocation, target)

        nearbyAllies ← number of allied robots within
radius 2 of newLocation

        score ← score + (nearbyAllies * 3)

        IF score < bestScore
            bestScore ← score
            bestDirection ← direction

    END FOR

    IF bestDirection exists
        move(bestDirection)
```

```
FIND_CLOSEST_RUIN

    closestRuin ← null
    minDistance ← infinity

    nearbyTiles ← sense nearby map tiles

    FOR each tile in nearbyTiles

        IF tile does not contain ruin
```

```
            CONTINUE

        robot ← robot at ruin location

        IF robot exists AND robot is a tower
            CONTINUE

        distance ← distanceSquared(myLocation,
ruinLocation)

        IF distance < minDistance
            minDistance ← distance
            closestRuin ← ruinLocation

    END FOR

    RETURN closestRuin
```

```
FIND_BEST_SPLASH_TARGET

    bestTarget ← null
    bestScore ← 0

    FOR dx = -3 to 3
        FOR dy = -3 to 3

            target ← myLocation + (dx,dy)

            IF target not on map
                CONTINUE

            score ← EVALUATE_SPLASH(target)

            IF score > bestScore
                bestScore ← score
                bestTarget ← target

        END FOR
    END FOR

    IF bestScore > threshold AND robot can attack
bestTarget
        attack(bestTarget)
```

```
EVALUATE_SPLASH(center)
```

```
    score ← 0

    FOR dx = -1 to 1
        FOR dy = -1 to 1

            tile ← center + (dx,dy)

            IF tile outside map
                CONTINUE

            paint ← tile paint

            IF paint is enemy
                score += 10

            ELSE IF paint empty
                score += 1

        END FOR
    END FOR

    RETURN score
```

```
FIND_CLOSEST_ENEMY_PAINT

    closestTile ← null
    minDistance ← infinity

    tiles ← sense nearby tiles

    FOR each tile in tiles

        IF tile paint is not enemy
            CONTINUE

        distance ← distanceSquared(myLocation, tile)

        IF distance < minDistance
            minDistance ← distance
            closestTile ← tile

    END FOR

    RETURN closestTile
```

b. Bot “Laliro”

i. Logika Soldier

```
SOLDIER
    // 1. Greedy Priority 1: Refill jika cat < 30%
    if (Paint < 30% capacity) then
        moveToNearestAllyTower()
        refillPaint
        return
    end if

    // 2. Greedy Priority 2: Bangun Reruntuhan (Ruins)
    if (findNearbyRuin) then
        buildTower(ruinLoc)
        return
    end if

    // 3. Greedy Priority 3: Selesaikan SRP (Special Resource
Pattern)
    if (canCompleteResourcePattern()) then
        completeResourcePattern()
        return
    end if

    // 4. Greedy Priority 4: Cari dan warnai ubin musuh/kosong
    scoreAndMoveToBestPaintTarget
```

ii. Logika Tower

```
TOWER
    attackLowestHPEnemy()

    // Spawn berdasarkan round
    if (round < 200) spawn(65% Soldier, 25% Mopper, 10% Splasher)
    else if (round < 800) spawn(35% Soldier, 35% Splasher, 30%
Mopper)
    else spawn(20% Soldier, 50% Splasher, 30% Mopper)

    // Spawn preference: mendekati tengah peta
    dirs = getOrderedDirections(towardCenter)
    for dir in dirs: if (canBuild) build(dir)
END
```

iii. Logika Mopper

```
procedure runMopper(rc)
    // Berikan cat ke rekan atau menara yang kehabisan stok
    neediestAlly <- findAllyWithLowestPaint(radius=2)
    if (neediestAlly exists and myPaint > 50%) then
        transferPaint(neediestAlly, amount)
        return

    // Mop Swing jika ada banyak musuh berderet
    bestSwingDir <- getBestMopSwingDirection()
    if (bestSwingDir exists) then
        mopSwing(bestSwingDir); return

    // Hapus cat musuh di sekitar
    enemyPaint <- findNearestEnemyPaint()
    if (enemyPaint exists) then
        if (isAdjacent(enemyPaint)) then
            attack(enemyPaint)
        else
            moveToward(enemyPaint)
        return

    // Ikuti Soldier terdekat jika tidak ada target cat
    soldier <- findNearestAllySoldier()
    if (soldier exists) then
        moveToward(soldier)
    else
        exploreZones()
```

iv. Logika Splasher

```
SPLASHER(rc)
    // kembali ke menara jika cat < 25%
    if (Paint < 30% capacity) then
        moveToNearestAllyTower()
        refillPaint
        return

    // Cari titik serangan area dengan nilai tertinggi
    bestSplashPos <- findBestSplashTarget(score_threshold)
    if (bestSplashPos exists) then
        attack(bestSplashPos)
        return
    end if

    // Cari area yang belum terwarnai dengan bias target zona
    target <- findBestUnpaintedTile
    moveToward(target)
```

v. Logika Utils

```
NAV
    if (canMove) then
       move
    else
       tryRotatedDirections(target)

EXPLORE
    if (targetReached or turn mod 50 == 0) then
       updateGlobalTarget()
    nav(globalTarget)
```

c. Bot "MyBotGweh"

1. Soldier

```
SOLDIER_RUN
    update stuck state (lastLoc, stuckTurns)

    IF needsRefill() THEN
        refillPaint()          // prioritas Paint Tower, fallback
tower lain
        paintCurrentTile()
        RETURN

    IF attackEnemyTower() success THEN
        RETURN

    moveToNearestUnpainted()   // memakai findBestTarget()
    paintCurrentTile()
```

```
SOLDIER_FIND_BEST_TARGET(exclude)

    nearbyTiles <- senseNearbyMapInfos()
    best <- null

    IF mapArea >= 1600 THEN
        allies <- senseNearbyRobots(ally)
        bestScore <- -infinity

        FOR each tile in nearbyTiles:
```

```
            skip if not passable
            skip if tile == exclude
            priority <- 2 if enemy paint, 1 if empty, else 0
            skip if priority == 0

            score <- priority*100 - dist(myLoc,tile)

            // bonus arah agar pergerakan stabil
            IF tile direction selaras lastMoveDir (+left/+right):
                score += 30

            // penalti kerumunan ally
            score -= countAlliesNear(tile, radius=2)*20

            choose max score
    ELSE
        // small/medium: pilih priority tertinggi, tie-break
jarak terdekat
        choose tile by (priority desc, distance asc)

    RETURN best


SOLDIER_NEEDS_REFILL

    mapArea <- mapWidth * mapHeight
    threshold <- 0.42 if large map, else 0.25
    RETURN paint < paintCapacity * threshold
```

2. Mopper

```
MOPPER_RUN
    update stuck state

    IF transferToAlly() THEN RETURN         // prioritas 1
    IF mopSwing() THEN RETURN               // prioritas 2
    IF mopEnemy() THEN RETURN               // prioritas 3
    IF moveToEnemyPaint() THEN RETURN       // prioritas 4

    followSoldier()                         // prioritas 5


MOPPER_MOP_SWING
    FOR each cardinal direction (N,E,S,W):
        if canMopSwing(dir):
            count enemies near step1 or step2 in that direction
    pilih direction dengan enemy terbanyak
    swing jika maxEnemies > 0
```

3. Splasher

```
SPLASHER_RUN
    update stuck state

    IF needsRefill() THEN
        refillPaint()
        RETURN

    IF splash() success THEN
        RETURN

    moveToUnpaintedArea()   // pakai currentTarget +
findBestTarget + fallback explore


SPLASHER_SPLASH
    nearbyTiles <- senseNearbyMapInfos()
    bestTarget <- null
    bestScore <- 0

    FOR each tile in nearbyTiles:
        skip if dist(myLoc,tile) > 4
        skip if cannot attack tile
        score <- countSplashValue(tile center)
        choose max score

    minScore <- 8 if large map else 4
    IF bestTarget exists AND bestScore >= minScore:
        attack(bestTarget)
        return true
    return false


SPLASHER_COUNT_SPLASH_VALUE(center)
    score <- 0
    FOR each sensed tile:
        IF dist(center, tile) <= 2:
            +3 if enemy paint
            +2 if empty
    RETURN score
```

4. Tower

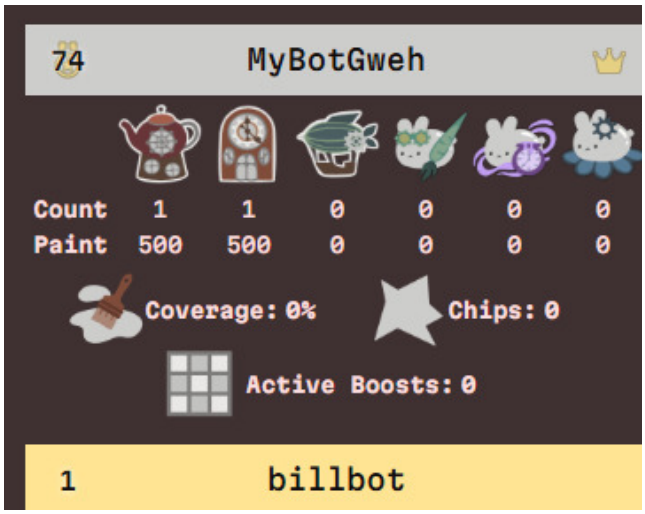
```
MONEY_TOWER_RUN
    attackEnemy()   // pilih musuh hp terendah
```

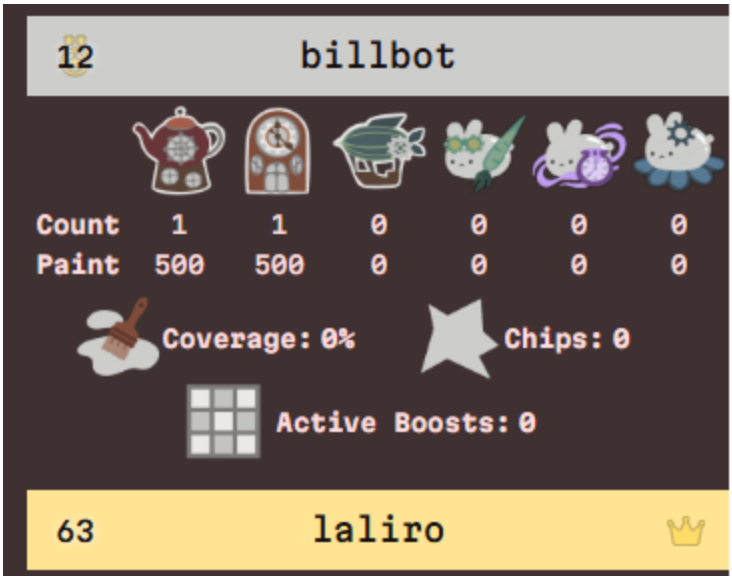
```
    spawnUnit()

        skip if cannot attack tile
        score <- countSplashValue(tile center)
        choose max score

    minScore <- 8 if large map else 4
    IF bestTarget exists AND bestScore >= minScore:
        attack(bestTarget)
        return true
    return false


MONEY_TOWER_ATTACK_ENEMY
    enemies <- senseNearbyRobots(opponent)
    if empty return
    weakest <- enemy dengan health minimum
    if canAttack(weakest.loc): attack(weakest.loc)

MONEY_TOWER_SPAWN
    mapArea <- mapWidth*mapHeight

    IF mapArea >= 1600:
        ratio Soldier:Mopper:Splasher = 2:1:2 (mod 5)
    ELSE IF mapArea >= 900:
        ratio Soldier:Mopper:Splasher = 3:1:1 (mod 5)
    ELSE:
        ratio Soldier:Mopper = 3:1 (mod 4)

    scan area spawn radius 2 (5x5)
    build first valid tile
    increment counter

PAINT_TOWER_RUN
    reuse attackEnemy() from MoneyTower
    spawnUnit() with fixed ratio 3:1:1 (Soldier:Mopper:Splasher)
```

Struktur Data Utama:

1. Navigasi dan anti-stuck:
   lastLoc, stuckTurns, lastMoveDir.
2. Targeting state:
   currentTarget, lastFailedTarget, exploreTarget.
3. Logistik refill:
   knownTowerLoc, estimateCoverage.
4. Eksplorasi berbasis zona/ID:
   cachedExploreOrder, cachedZoneOrder, exploreZoneIdx.
5. Counter produksi tower:
   soldierSpawned, mopperSpawned, splasherSpawned.

4.2. Analisis Hasil Pengujian




12
billbot
Count 1 1 0 0 0 0
Paint 500 500 0 0 0 0
Coverage: 0%
Chips: 0
Active Boosts: 0
63
laliro

Bab 5

Kesimpulan dan Saran

Algoritma *greedy* memiliki keuntungan tersendiri dalam pembuatan robot Battlecode 2025, seperti mengurangi *barrier-of-entry* ketika pertama kali membuat program dengan mendekomposisikan program menjadi beberapa bagian yang lebih *digestible*. Selain itu, algoritma *greedy* juga bisa digunakan sebagai *stepping stone* untuk membuat program yang optimal untuk melawan robot-robot lainnya.

Meskipun demikian, batasan penggunaan algoritma rasanya menghambat dalam pembuatan robot yang optimal. Betul, memang tujuan awal bukan untuk mencari pemenang, tetapi untuk membangun pengertian mahasiswa akan algoritma rakus dan tamak. Namun, alangkah baiknya jika masalah yang harus diselesaikan lebih *tailored* ke algoritma *greedy* sehingga memiliki *sense of accomplishment* tanpa ada pikiran "Harusnya bisa lebih bagus, tapi gak boleh".

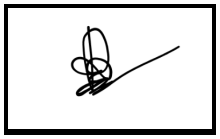
Lampiran

Github Repository: https://github.com/PikaProgram/Tubes1_BukaPuasaJamSatu/

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Billy Ontoseno Irawan

Muhammad Rafi Akbar



Rasyad Satyatma

Daftar Pustaka

Munir, R. Algoritma Greedy Bagian 1. Diakses pada 11 Maret 2026 dari : https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf

Munir, R. Algoritma Greedy Bagian 2. Diakses pada 11 Maret 2026 dari : https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag2.pdf

Munir, R. Algoritma Greedy Bagian 3. Diakses pada 11 Maret 2026 dari : https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag3.pdf

Geffner, I. A Guide to Battlecode. Diakses pada 13 Maret 2026 dari: https://battlecode.org/assets/files/battlecode-guide-xsquare.pdf

https://releases.battlecode.org/javadoc/battlecode25/3.1.0/battlecode/common/package-summary.html